

Visual Network Topology Design And Automated Deployment

GRS Project

José Lopes
up202208288

Pedro Marinho
up202206854

Overview

- Context & Problem
- Solution
- Architecture & Core Systems
- Evaluation
- Live Demonstration

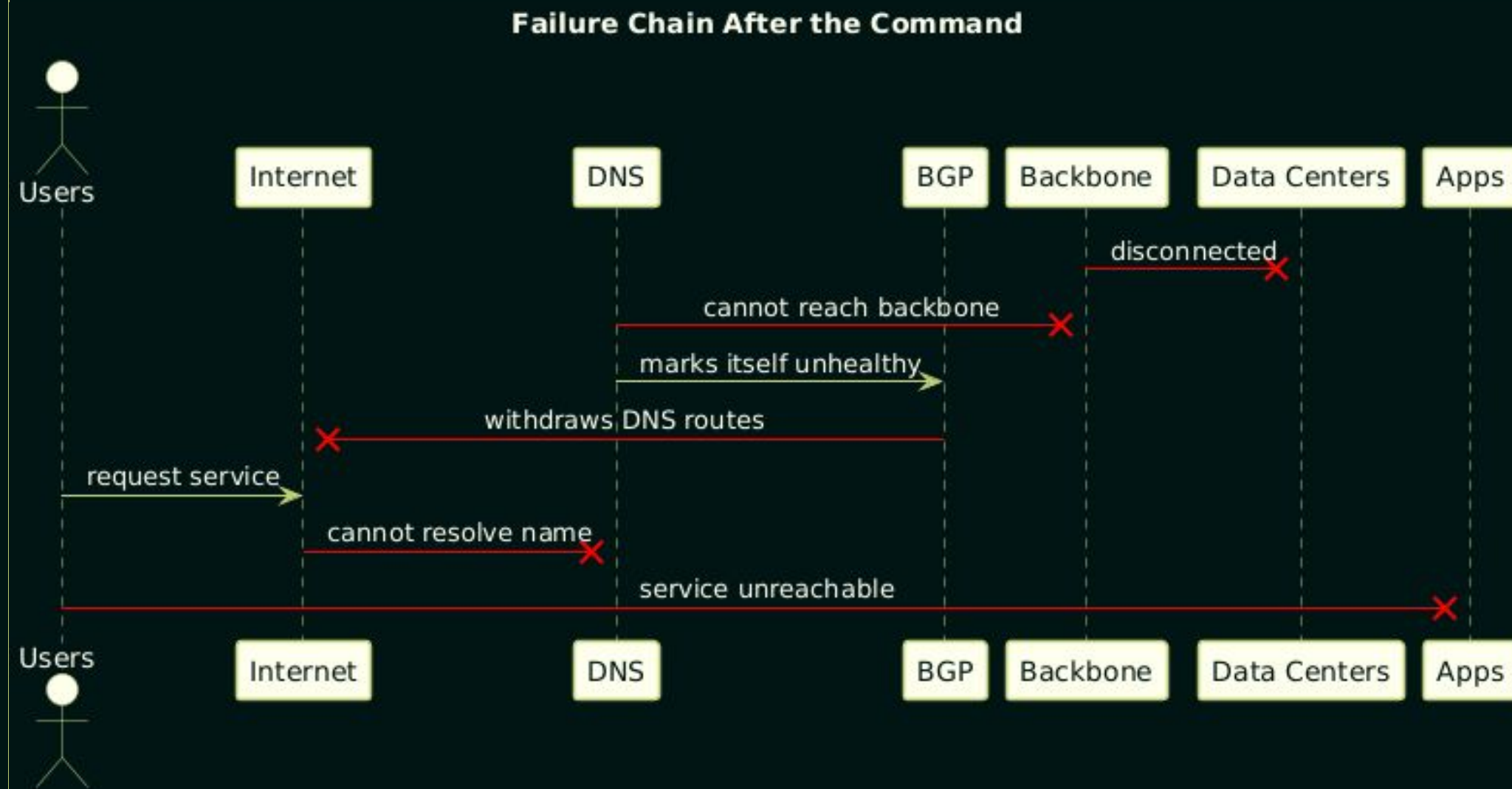
Motivations

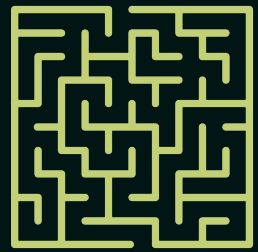
Our goal:

- Simulate networks in a controlled environment, test configuration changes safely, observe their impact, and only then deploy them to a real network.

Real example: [Facebook outage, October 2021](#)

- A backbone maintenance command unintentionally disconnected Facebook's data centers from the rest of the network.
- Because the DNS sites could no longer reach the internal backbone, they marked themselves as unhealthy and withdrew their BGP advertisements.
- The DNS servers were still running, but the Internet could no longer reach them. Without DNS, users could not find Facebook, Instagram, or WhatsApp.
- **Key idea:** a routing change can cascade into DNS failure and application downtime.





Networking is overwhelming for beginners



Manual setup is slow



Small IP/routing mistakes are easy to make breaking the whole lab



Dynamic routing and services are hard to configure consistently



Testing different topologies requires rebuilding many files

Problems

Solution

Intuitive drag-and-drop topology editor



Automatic Docker Compose and device configuration generation



Automatic defaults, validation, and generated router interfaces



Built-in support for OSPF, DNS, load balancing, and service templates



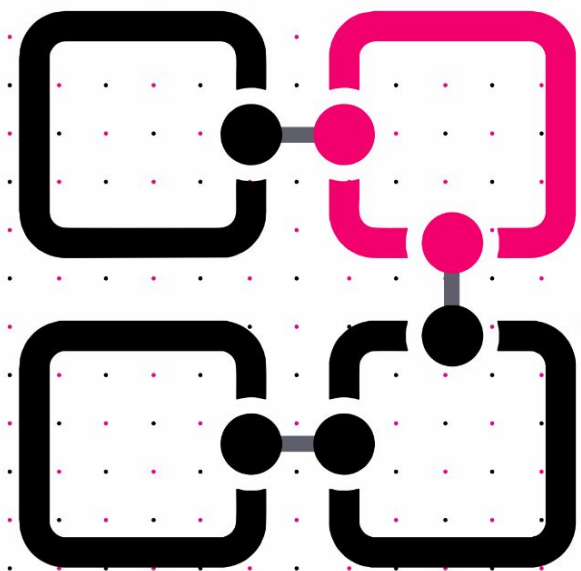
Import/export topology JSON and regenerate the full lab from one model



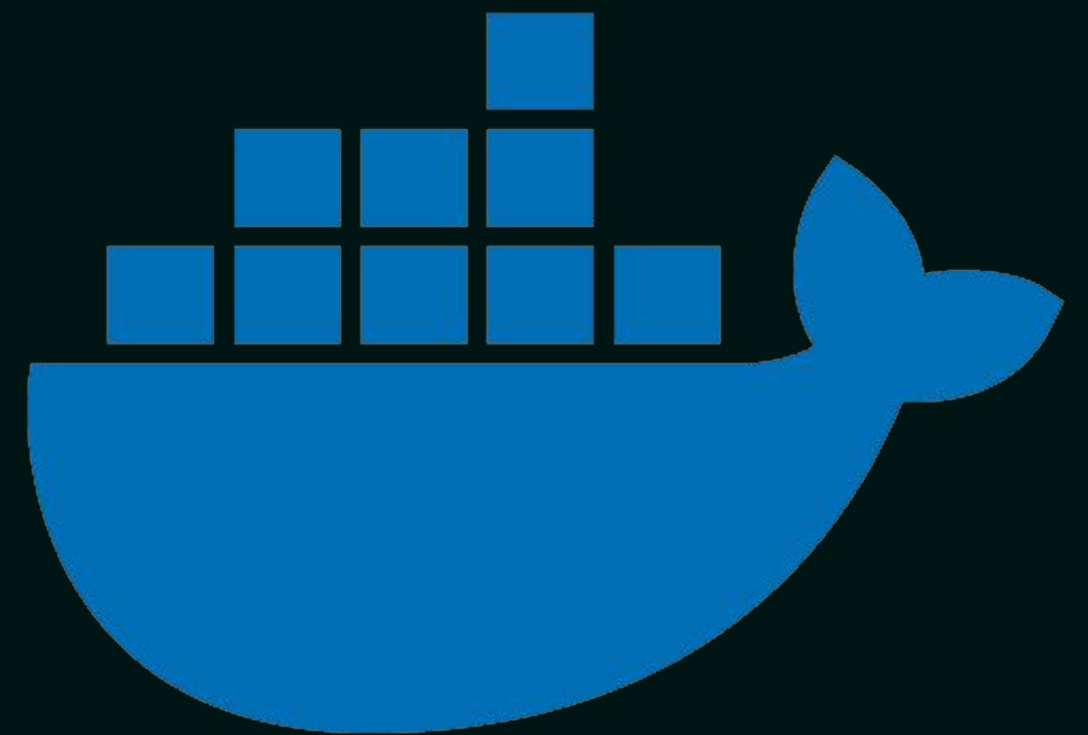
Technologies Used



FastAPI

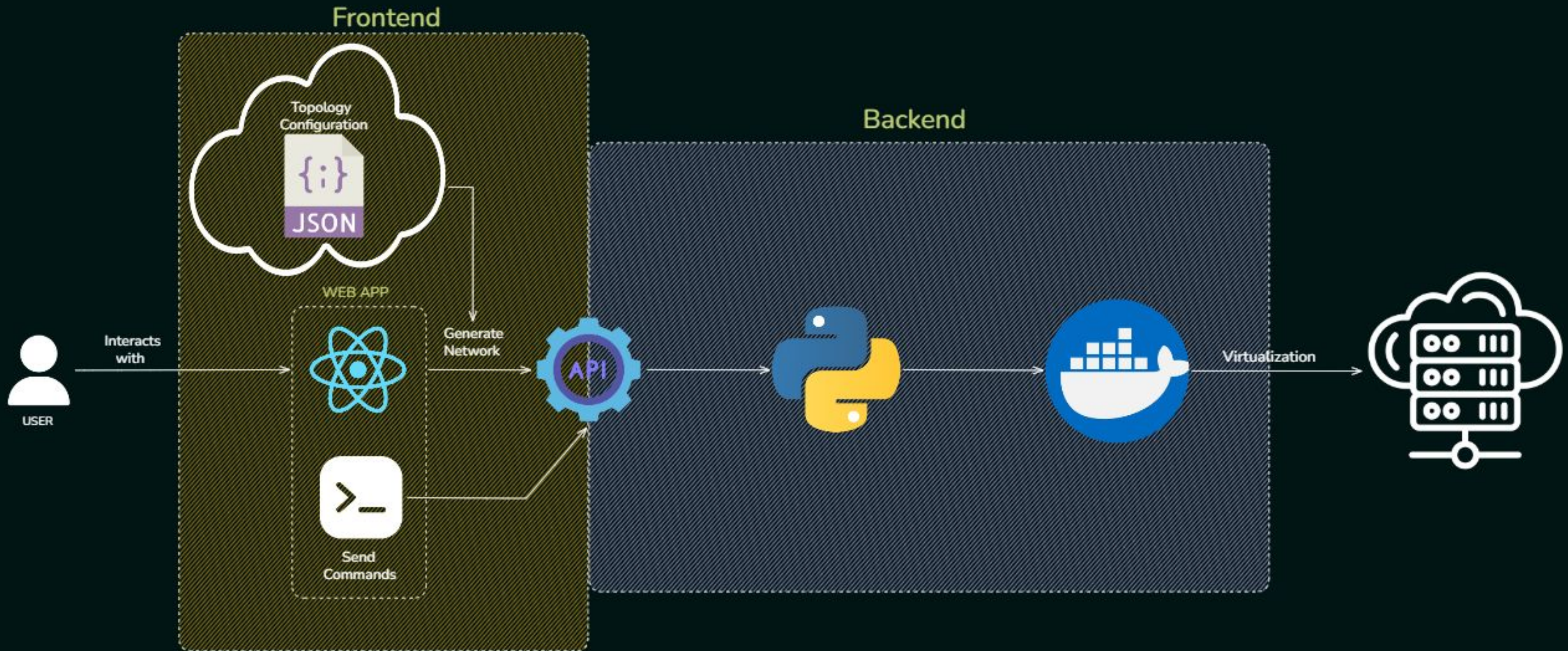


React Flow



docker

Architecture



Current Features

Visual topology editor

- Drag-and-drop networks, routers, hosts, servers, DNS and load balancers.

Automatic deployment

- Generates Docker Compose, device folders and startup scripts.

Routing support

- Static route generation and OSPF with configurable area and cost.

Network services

- DNS server with editable records, Nginx load balancer and HTTP servers.

Testing tools

- Interactive terminals and command execution inside containers.

Future Work

Topology analyzer

- Detect invalid IPs, missing gateways, unreachable DNS, broken routes, ...

More protocols and devices

- Add BGP, firewalls, NAT, VPNs and monitoring probes.

Readiness checks

- Automatically verify OSPF convergence, DNS resolution and service availability.

Experiment automation

- Allow running benchmark scripts inside hosts.

More deployment targets

- Support engines beyond Docker Compose, such as Kubernetes or network emulators.

Alternatives

GNS3

Powerful network emulator used for realistic router and firewall labs.

Supports real network images and complex professional scenarios.

Main limitation for our use case:

setup is heavier and devices still need to be configured mostly manually.

NetCompose focuses on faster visual topology creation and automatic Docker-based deployment.

EVE-NG

Web-based multi-vendor network emulation platform.

Very strong for advanced labs with routers, switches, firewalls and security appliances.

Main limitation for our use case:
more complex environment, usually aimed at professional training and certification labs.

NetCompose is simpler and more focused on educational experimentation and automatic service configuration.

Cisco Packet Tracer

Beginner-friendly simulator commonly used to learn basic networking concepts.

Good visual interface for IP addressing, routing and switching exercises.

Main limitation for our use case:

it is mostly a simulator and does not deploy real Docker services such as DNS, Nginx or custom servers.

NetCompose combines visual learning with runnable container-based services.

Conclusion

The development process deepened our understanding of IP addressing, routing, DNS, OSPF and service reachability.

Making the network actually run revealed practical issues that are easy to miss when studying only the theory.

The prototype can already be used as a visual learning lab for beginners.

With more devices and protocols, it could evolve into a realistic network emulator. Extending it is fairly straightforward because the code was designed with future expansion in mind.

Advanced users could use it to validate changes and test failure scenarios before deploying them in real networks.

Q&A + Live Demo

Any Questions ?

